



中国石油大学 (华东)
CHINA UNIVERSITY OF PETROLEUM

本科毕业设计（论文）

题 目：

基于 LayaAir 与 ECS 的 Roguelite 生存战斗游
戏

设计与实现

学生姓名：_____刘瀚文_____

学 号：_____2207020509_____

专 业：_____计算机科学与技术专业_____

班 级：_____软件工程 2205 班_____

指导教师：_____董玉坤_____

2026 年 6 月

学位论文原创性声明

本人所提交的学位论文《基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏设计与实现》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。本声明的法律后果由本人承担。

论文作者（签名）：_____ 指导教师确认（签名）：_____
年 月 日 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）：_____ 指导教师（签名）：_____
年 月 日 年 月 日

摘 要

随着 HTML5 与 WebGL 技术的发展,浏览器端已能承载中等复杂度的实时动作游戏。Roguelite 品类强调单局随机性、元进度与构筑深度,对架构可扩展性与运行时性能要求较高。本文以毕业设计 Survivor And Fight 为对象,在 LayaAir 3.x 与 TypeScript 上设计并实现 2D 俯视角生存战斗游戏。系统采用轻量实体组件系统解耦位置、属性、技能与操控,配表驱动发射、穿透、分裂与连锁等战斗效果;通过对象池降低实例化开销,并在高实体规模时以 Web Worker 异步重算战斗快照。

元游戏层提供开始界面、选关与三幕跑图及局内升级装配。全文从需求、架构、实现与测试验证展开。压力测试表明系统运行稳定,同屏千怪时对象池使 P95 帧时间由约 76.5 ms 降至 48.6 ms,动态图集提升平均帧率约 13.4%;Web Worker 同窗定量消融留作后续工作。

关键词: LayaAir; 实体组件系统; Roguelite; 数据驱动; Web Worker; 对象池; 生存战斗

Abstract

With the maturity of HTML5 and WebGL, browsers can host real-time action games of moderate complexity. This thesis presents the design and implementation of Survivor And Fight, a 2D top-down survival combat game built on LayaAir 3.x and TypeScript. A lightweight ECS drives gameplay; combat is configuration-driven with object pooling and optional Web Worker offloading. A meta layer provides menus, level selection, and a three-act DAG run map. Testing shows stable operation; object pooling reduces P95 frame time under 1000 on-screen monsters, and texture atlasing improves average FPS on the heaviest wave. Quantitative ablation of Worker offloading under warmed-up conditions is left for future work.

Keywords: LayaAir, ECS, Roguelite, data-driven, Web Worker, object pooling, survivor-like combat

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 游戏引擎架构	2
1.2.2 实体组件系统与玩法架构	2
1.2.3 物理模拟、人群与性能	2
1.3 项目背景	3
1.4 主要创新点与特色	3
1.5 伦理与合规说明	4
1.6 相关游戏与竞品简要分析	4
1.7 术语与缩写预备说明	5
1.8 本章小结	5
第 2 章 相关技术与理论基础	6
2.1 引擎语言选择	6
2.1.1 H5 游戏引擎选型	6
2.1.2 TypeScript 与工程常量管理	7
2.1.3 主循环、预制体与引擎使用约束	7
2.2 项目玩法架构选择	7
2.2.1 Survivor-like 品类与核心循环	7
2.2.2 轻量实体组件系统 (ECS)	8
2.2.3 数据驱动与 JSON 配表	8
2.2.4 Roguelite 跑图与关卡结构	9
2.2.5 战斗性能与密集实体仿真	9
2.3 UI 架构选择	10
2.3.1 MVC 分层与全屏界面栈	10
2.3.2 战斗内 UI: HUD、暂停与技能装配	10
2.3.3 UI 架构选型的综合评价	11
2.4 本章小结	11

第 3 章 系统需求分析	12
3.1 可行性分析	12
3.1.1 技术可行性	12
3.1.2 经济与社会可行性	12
3.2 用户角色与运行环境	12
3.2.1 用户角色	12
3.2.2 运行环境	13
3.3 功能性需求	13
3.3.1 用例：局内战斗	13
3.3.2 用例：元游戏进入战斗	14
3.4 功能需求详细说明	14
3.4.1 ECS 核心	14
3.4.2 战斗与技能	14
3.4.3 怪物、元游戏与 UI	14
3.5 非功能性需求	14
3.6 数据、接口与约束	15
3.7 需求优先级	15
3.8 需求与测试映射	15
3.9 本章小结	15
第 4 章 系统总体设计	17
4.1 设计原则	17
4.2 系统具体设计	17
4.2.1 基于 ECS 的 Gameplay 实现	17
4.2.2 基于 MVC 的 UI 结构实现	18
4.2.3 基于 Web Worker 和对象池的优化实现	19
4.3 系统交互与时序	19
4.4 可靠性设计	19
4.5 容错与降级策略	21
4.6 可扩展性设计	21
4.7 本章小结	22

第 5 章 核心模块详细设计与实现	23
5.1 ECS 架构 Gameplay 实现	23
5.1.1 世界、组件与筛选器	23
5.1.2 属性、移动与自动施法	23
5.1.3 技能与效果执行链	23
5.1.4 子弹碰撞与怪物 AI	24
5.1.5 经验、升级与元游戏数据	24
5.2 MVC 架构 UI 实现	24
5.2.1 UI 栈与元游戏界面	24
5.2.2 战斗 HUD 与技能装配	24
5.2.3 输入与重启面板	24
5.3 Web Worker 和对象池的优化实现	25
5.4 游戏主循环设计	25
5.4.1 入口与配置时序	25
5.4.2 SimpleEcsDemo 构造与 init	25
5.4.3 帧循环与暂停	25
5.5 典型场景实现剖析	26
5.5.1 场景一：玩家首次进入战斗	26
5.5.2 场景二：Tab 打开技能面板并更换装备	26
5.5.3 场景三：怪物密集时的 Worker 卸载	26
5.6 关键代码文件索引	26
5.7 功能模块与论文章节对照	27
5.8 本章小结	27
第 6 章 性能优化与工程实践	28
6.1 性能目标与瓶颈	28
6.2 对象池	28
6.3 Web Worker 卸载	28
6.4 渲染与逻辑侧其它优化	29
6.5 帧预算与内存	29
6.6 性能实验设计	29
6.6.1 实验目的与变量	29
6.6.2 实验步骤	29

6.6.3 优化自检清单	30
6.7 绿色计算与兼容性	30
6.8 本章小结	30
第 7 章 系统测试与验证	31
7.1 测试策略	31
7.2 单元验证	31
7.2.1 ECS 核心	31
7.2.2 公式解析器	31
7.2.3 属性修饰器	31
7.3 功能测试	32
7.3.1 集成测试补充	32
7.4 性能测试	32
7.4.1 测试环境	32
7.4.2 动态图集对比（前三波）	33
7.4.3 五波完整帧率	33
7.4.4 对象池消融（第 4-5 波）	33
7.5 系统运行截图	34
7.6 测试结论	34
7.7 本章小结	38
第 8 章 总结与展望	39
8.1 工作总结	39
8.2 不足与展望	39
8.3 社会、健康与文化影响	40
8.4 绿色节能与兼容性	40
8.5 本章小结	40
致谢	41
附录	42
附录 A 名词术语及缩略词	43
附录 B 核心配表字段示例（节选）	44

附录 C 需求与测试用例映射（节选）	45
附录 D 缺陷记录（项目实测）	46

插图

4-1	系统分层架构	18
4-2	系统主要交互时序 (Mermaid 序列图)	20
4-3	系统主要交互时序 (文字摘要)	20
7-1	游戏开始界面 (StartPanel)	35
7-2	三大关 DAG 跑图 (RunMapPanel)	35
7-3	局内战斗 (同屏多怪与弹幕)	36
7-4	Tab 暂停下的技能/Effect 装配面板	36
7-5	生存胜利后的三选一奖励面板	37
7-6	玩家死亡后的重启面板	37

表格

2-1	主流 H5/跨平台方案与本项目选型对比	6
2-2	Unity DOTS 与项目轻量 ECS 对比	8
2-3	UI 架构选型要点汇总	11
3-1	运行环境建议配置	13
3-2	主要功能性需求一览	13
3-3	非功能需求量化指标（验收参考）	15
3-4	需求优先级（MoSCoW）	15
3-5	需求与测试映射（节选）	16
4-1	容错与降级策略	22
5-1	效果类型与执行器映射	23
5-2	核心源码文件索引	26
5-3	功能模块与论文章节对照	27
7-1	功能测试用例摘要	32
7-2	测试环境配置	33
7-3	测试关卡分波 FPS（接入动态图集，前三波）	33
7-4	动态图集接入前后 FPS 对比（前三波）	33
7-5	Level3 五波测试帧率（单次运行）	34
7-6	性能对比实验（对象池消融，1000 怪 / 10s）	34
C-1	需求—测试映射（节选）	45
D-1	缺陷记录表	46

第 1 章 绪论

1.1 研究背景与意义

移动互联网与浏览器平台普及后，H5 游戏成为独立开发与教学实践常用的载体。相对原生客户端，H5 在分发、跨平台与迭代上成本更低：玩家通过链接进入，无需安装；开发者可用 Web 技术栈统一维护逻辑与资源。另一方面，JavaScript 单线程事件循环、垃圾回收（GC）抖动以及浏览器渲染管线会限制实时战斗类游戏的架构设计——单帧内逻辑、物理近似、碰撞与 UI 事件一旦叠加过多，就会表现为卡顿或掉帧。

Roguelite 以自动攻击、大量敌人、局内升级构筑为核心循环，对数据驱动、同屏实体规模与元游戏流程提出工程要求。与回合制 Roguelike 相比，Roguelite 更强调单局 Build 与节奏压缩，适合在毕业设计周期内交付可玩闭环；与纯 Arcade 射击相比，随机奖励与路线选择要求架构便于调参与扩展。Survivor-like 将操作简化为走位与构筑，技术难点集中在高密度实体模拟与技能效果组合，而非复杂输入。

Survivor And Fight 在 LayaAir 3.x 上实现 2D 俯视角生存战斗原型，覆盖主菜单、选关、爬塔式跑图、局内战斗、升级奖励与技能装配等流程，并采用 ECS、配表驱动技能、对象池与 Web Worker 等性能手段。游戏引擎子系统多、耦合高，维护与演化成本是常见问题^[?, ?, ?]。Ullmann 等提出的 SyDRA 通过恢复子系统依赖关系辅助架构理解与变更影响分析^[?]；Gregory 对渲染、资源、场景图与脚本分层的论述，为本文“表现层—逻辑层—基础设施层”划分提供参照^[?]。

Unity 等引擎上已有基于物理模拟的人群疏散验证^[?, ?]，说明重计算与渲染分离、模块化设计的必要性。Cantrell 等在 Unity 中实现基于物理压力的人群疏散；Petty 等强调仿真系统须经 V&V（验证与确认）^[?]。本项目不做物理引擎级仿真，但怪物追逐、分离力与密集碰撞同属多代理体局部相互作用，可借鉴分阶段验证与性能—真实性折中思路。在 H5 环境下实践上述思想，既有工程价值（可运行原型与性能数据），也有教学价值（贯通需求、设计、实现、测试）。

1.2 国内外研究现状

1.2.1 游戏引擎架构

游戏引擎研究长期关注子系统划分、依赖与演化维护。Gregory 从工业实践归纳渲染、资源、场景图、动画与脚本等职责边界，指出商业引擎将 tick 调度与资源生命周期统一管理^[7]。Munro 对 Quake 系列、Agrahari 对 Unreal 内部分析，为“引擎层—游戏层”分界提供历史参照^[7, 7]。Goslin 介绍的 Panda3D 体现脚本语言与 C++ 渲染核心协同的典型模式^[7]。

近年研究更多面向架构理解与变更影响。Ullmann 等提出 SyDRA，恢复多款开源引擎子系统依赖图，辅助理解任务与影响分析^[7]；后续工作可视化子系统耦合，指出过度耦合与目录嵌套是常见退化^[7]。Baabad 等从软件退化角度讨论大型引擎代码库维护成本^[7]。H5 与跨平台方向除传统子系统外，还须考虑主线程预算、资源异步加载与浏览器沙箱。LayaAir、Cocos Creator 等一体化 H5 引擎降低 2D 预制体与 TypeScript 工作流成本，但逻辑与表现是否解耦仍取决于应用层架构^[7, 7]。

1.2.2 实体组件系统与玩法架构

实体组件系统（ECS）用实体标识、纯数据组件与无状态系统表示游戏对象，强调组合优于继承，便于按类型批量处理与扩展^[7]。Unity DOTS 结合 Job System 与 Burst，面向大规模数据导向并行；独立游戏与工具链社区亦讨论 ECS 在各类引擎中的落地。教学与中小型项目多采用 Map 存组件、单线程顺序调度的轻量实现，在可读性与性能之间折中，避免完整 DOTS 的学习与调试成本。

玩法上，Survivor-like 强调波次刷怪、自动输出与局内升级；Roguelite 在单局随机外引入元进度与路线选择。Slay the Spire 式有向无环图（**DAG**）跑图可支撑路线规划^[7]。数据驱动技能表在 RPG 与动作游戏中已较普遍：冷却、伤害与效果段外置为表，由运行时解释；相关研究也关注公式解析的安全边界，避免不可信配置触发任意代码^[7, 7]。

1.2.3 物理模拟、人群与性能

大量代理体运动与密集交互是生存战斗类游戏的常见瓶颈。Reynolds 的 Boids、Thalmann 与 Pelechano 的人群仿真为局部规则驱动群体运动提供经典范式^[7, 7, 7]。Cantrell 等在 Unity 上实现物理压力人群疏散并完成 V&V^[7]；Bott 等在 Unreal 中采用物理驱动人群工具链^[7]；Helbing 等关于恐慌人群模型提示密集场景须关注分离

力与稳定性^[7]。McKenzie 等将人群行为建模与游戏技术结合用于仿真教学^[7]。

H5 客户端性能研究多关注主线程敏感性与 GC。工程上常用对象池抑制反复实例化、纹理合批与图集、逻辑暂停，以及将纯数值计算迁至 Web Worker^[7, 8]。Worker 通信成本与 DOM/引擎 API 不可达性决定哪些逻辑可卸载、哪些须留主线程，是浏览器游戏性能优化中的常见结论。

1.3 项目背景

Survivor And Fight 是面向浏览器的 2D 俯视角 Roguelite 生存战斗原型，使用 LayaAir 3.x 与 TypeScript 开发。项目要在可运行工程上形成主菜单与选关、爬塔式 **DAG** 跑图、局内自动战斗、升级奖励与 **Tab** 技能装配等玩法闭环，并检验轻量 ECS、JSON 配表技能、对象池与 Web Worker 在 H5 下的可行性。

产业侧，国产 H5 与小游戏平台对即点即玩、低安装成本作品的需求在增长。Survivor-like 操作少、同屏敌人多；Roguelite 用单局随机与路线选择提高重玩性。二者叠加可在有限美术与关卡人力下较快迭代，适合本科毕业设计周期。本项目不以内内容体量对标商业大作，而以可扩展客户端架构与可复现性能数据为主，为后续玩法、技能与关卡留接口。

技术侧，H5 实时战斗受 JavaScript 单线程、GC 与渲染预算约束。同屏数百怪物与大量弹幕时，若每帧 `instantiate/destroy` 预制体、在主线程完成全部追逐与碰撞，帧时间易出现尖峰。选型因此强调：逻辑与表现分离（ECS + Laya 节点）、参数外置（配表 + `defines`）、可降级优化（对象池、逻辑暂停、Worker 回退）。引擎子系统耦合与演化在文献中已有讨论^[7, 8, 9]；人群与多代理体运动研究为怪物追逐、分离力与密集碰撞简化提供参照^[7, 8, 9]。

仓库已有可编译的 LayaAir 工程、`docs/config/` 下技能/子弹/角色等 JSON 配表及部分 2D 资源。论文工作是对实现的需求归纳、架构说明、模块描述、测试验证与性能分析，使 Demo 成为可评阅、可答辩的毕业设计成果。

1.4 主要创新点与特色

本项目具有以下工程特色：

- (1) 轻量 **ECS** 与引擎解耦：在 LayaAir 上实现实体—组件—系统内核，玩法逻辑由 `EcsWorld` 调度，`ViewComponent` 只做节点绑定与同步，减少业务与引擎 API 缠绕。

- (2) 配表驱动的技能效果链：技能拆为 Skill 与多行 SkillEffect，修饰器与子弹效果按序组合，支持穿透、分裂、连锁等；策划可改冷却、伤害与效果段而少动核心代码，当前配表含 70 余条效果配置。
- (3) 元游戏与局内战斗联动：三大关 DAG 跑图、节点类型（战斗/Boss/休息等）经 MetaFlowController 与战斗入口载荷联动；Boss 关可携带更长生存胜利时间等参数。
- (4) 同屏规模下的性能手段：BulletPool、MonsterPool 抑制实例化尖峰；实体数达标时快照经 Web Worker 计算追逐与分离，主线程保留子弹碰撞与渲染；实测中对象池可降低 P95 帧时间，动态图集在千怪波次下提高平均 FPS。
- (5) 可验证的 UI 与战斗协同：全屏界面经 UIStackManager 路由；Tab 打开技能装配时 GameSession.paused 暂停战斗，SkillLoadoutSyncSystem 同步面板与自动施法，便于手工回归。

上述特色可在运行原型与第 7 章测试数据中验证，并与第 4 章分层设计、第 5 章实现对应。

1.5 伦理与合规说明

游戏为虚拟战斗与生命值变化，不涉及真实暴力。项目不采集用户个人敏感信息。若公网部署，须遵守网络游戏管理与未成年人保护相关规定，标注适龄提示与游戏时长建议^[7]。论文第 8 章对社会影响有补充讨论。

1.6 相关游戏与竞品简要分析

吸血鬼幸存者（Vampire Survivors）：极简操作、自动攻击、海量敌人，验证了 Survivor-like 品类^[7]。本项目借鉴其战斗节奏与升级构筑，用 ECS 与 JSON 配表扩展逻辑，便于在论文中说明模块边界与数据流。

杀戮尖塔（Slay the Spire）：卡牌 Roguelike 与节点地图结合，强调路线与风险收益^[7]。本项目跑图采用类似 DAG，但战斗为实时俯视角，难点在帧率与碰撞而非回合状态机。

Noita：法术模块组合带来高自由度构筑^[7]。本项目 effect 链式修饰器（修饰器须在子弹效果之前）吸收组合深度思路，未实现完整物理与像素级交互，以控制毕业设计范围。

其他 **H5** 生存类作品：多使用 Cocos/Laya 预制体与传统面向对象结构。本项目以轻量 ECS、配表与 Worker 为差异化路径，侧重架构可维护性与性能可测性，不与商业产品在内容量上对标。

1.7 术语与缩写预备说明

全文首次出现英文缩写时给出中文释义，见附录 A。引擎 API 与核心类名保留英文，便于对照源码。

1.8 本章小结

本章说明课题背景、国内外相关研究与工程特色，为后续需求、设计与实现章节提供依据。

第 2 章 相关技术与理论基础

本章从引擎与语言、玩法与逻辑架构、界面架构三条线说明 Survivor And Fight 的技术选型及依据，为第 3 章需求与第 4 章设计提供共同语境。各节在文献与业界实践基础上给出毕业设计规模下的取舍，并与后文 SimpleEcsDemo、UIStackManager 等模块对应，避免孤立罗列概念。

2.1 引擎语言选择

2.1.1 H5 游戏引擎选型

浏览器分发的 2D 游戏常见两类路线：基于 Canvas/WebGL 的轻量库（Phaser、PixiJS），以及一体化引擎（Cocos Creator、LayaAir）。前者灵活，但场景管理、预制体、UI 与资源打包常需自行拼装；后者提供 IDE、预制体实例化、帧循环与输入封装，更适合在有限周期内交付可演示闭环^[7, 8]。

本项目选用 **LayaAir 3.x**，理由如下：（1）课程与实验室已有 Laya 工程经验，环境搭建成本低；（2）3.x 对 TypeScript 支持与本项目语言一致；（3）2D 预制体、Area2D 与 UI2（GBox、GButton 等）可满足元菜单、跑图与战斗 HUD；（4）内置预览与发布至 bin/ 便于性能测试与答辩演示。Gregory 指出商业引擎将渲染、资源、场景更新与脚本生命周期统一管理^[7]；LayaAir 在 H5 场景下承担类似运行时平台的角色。

相对 Unity、Unreal 等原生引擎，H5 分发成本更低，但受浏览器单线程与 GC 约束更明显。Ullmann 等对引擎子系统耦合的研究表明，无论引擎种类，划分“引擎层—游戏逻辑层”都有利于维护^[7]。因此 Laya 节点仅作表现层，核心玩法放在自研轻量 ECS 中更新，而非全部写入 Laya.Script 回调。

表 2-1 主流 H5/跨平台方案与本项目选型对比

方案	优势	劣势	本项目
Phaser/PixiJS	轻量、社区活跃	工作流需自建	未采用
Cocos Creator	生态成熟、工具链完整	与现有课设栈不一致	未采用
LayaAir 3.x	IDE 一体、TS 原生、2D/UI2 完备	文档与社区小于头部引擎	采用

2.1.2 TypeScript 与工程常量管理

脚本语言选用 **TypeScript**，在 JavaScript 上提供静态类型与模块化，实体、组件、系统增多时接口更稳定。Briand 等关于面向对象设计可维护性的实验表明，清晰模块边界与一致命名有助于降低理解成本^[7]。可调战斗常量、UI 阈值、路径正则等静态配置集中在 `src/defines/`，经 `index.ts` 统一导出，避免业务文件顶部散落魔法数；策划数值外置 JSON 配表（见第 2.2 节），区分“工程常量”与“策划参数”。

2.1.3 主循环、预制体与引擎使用约束

主循环由 `Laya.timer.frameLoop` 驱动：每帧先 `EcsWorld.update(deltaTime)`，再由引擎渲染并分发 UI 事件，符合 Gregory 所述逻辑 tick 与绘制分离的常见做法^[7, 8]。入口 `Main.ts` 挂在 `Area2D` 子节点而非 3D 根节点，避免纯 2D Demo 进入 3D 渲染管线。

使用 Laya API 须统一 `Laya.` 前缀（如 `Laya.Sprite`、`Laya.Handler.create`）。角色与子弹经 `Laya.Prefab.instantiate` 生成。可加载资源（材质、贴图、预制体）须放在 `assets/`，不可放在 `src/`，否则预览路径解析失败。2D 位移须用 `transform.position = new Laya.Vector3(...)` 或 `translate`，不可对 `position` getter 返回的副本调用 `setValue`，否则逻辑坐标已变而画面不动——该问题在相机跟随与怪物追逐调试中多次出现，本文记录以供规避。

2.2 项目玩法架构选择

2.2.1 Survivor-like 品类与核心循环

Survivor-like 核心循环可写为：移动躲避 → 自动输出伤害 → 击杀获经验 → 升级强化 *Build* → 敌人密度上升。相对传统射击，玩家负担从瞄准转向走位与构筑^[9]。客户端架构须同时满足：（1）持续移动时碰撞与伤害结算稳定；（2）数分钟内多次升级，奖励能写入属性或技能栏；（3）技能数值可通过配表调整，少改核心战斗代码。

`SimpleEcsDemo` 默认创建玩家并由 `PlayerAutoCastSystem` 自动施法，`ControlSystem` 处理走位，以降低操作门槛、突出构筑。经验由 `ExperienceSystem` 处理；击杀奖励与等级加成通过 `defines` 中 `MONSTER_EXP_REWARD_BASE`、`MONSTER_EXP_REWARD_LEVEL_BONUS` 等常量调节，属于可配置节奏而非硬编码。

2.2.2 轻量实体组件系统（ECS）

实体组件系统（ECS）将对象拆为实体标识（Entity）、纯数据组件（Component）与无状态系统（System）。新增怪物行为时往往只需增 System 或扩展组件字段，无需维护大继承树；按类型批量处理更方便^[7]。

Unity DOTS 采用 ECS + Job System + Burst，面向数万实体并行^[7]。毕业设计周期与团队规模有限，完整 DOTS 等价物学习与 Laya 节点绑定成本过高。本项目采用教学向轻量 ECS：EntityId 为数值；ComponentStore 按组件类型用 Map 存储；EcsWorld.update 单线程顺序调用已注册系统，按 input/logic/physics/render 分组排序。第 1 期目标是结构清晰、支撑数百同屏实体，不追求极致数据导向布局。

表 2-2 Unity DOTS 与项目轻量 ECS 对比

维度	Unity DOTS	本项目 ECS
并行模型	Job System 多线程	单线程；追逐等片段可选 Web Worker
内存布局 与引擎绑定	Archetype / Chunk Entities Graphics	按组件类型的 Map ViewComponent 绑定 Laya 节点
适用规模 迭代与调试成本	数万实体级 高	数百实体级 中低

玩法层组件含 PlayerTag/MonsterTag、Position/Velocity、Attribute（含可溯源 modifier）、Skill、SkillLoadoutState 等；系统含 MovementSystem、AttributeSystem、SkillSystem、BulletSystem、MonsterChaseSystem 等。FilterRegistry 提供 Players、Monsters 等命名筛选，避免各 System 重复写交集查询。怪物与玩家共享位移、属性、技能等组件，仅标签不同，体现组合优于继承。

2.2.3 数据驱动与 JSON 配表

Roguelite 与动作游戏普遍数据驱动：冷却、伤害、子弹参数、效果链外置为表，运行时加载^[7]。常见做法包括：（1）运行时加载 JSON 并反射填充数据类；（2）构建期生成 TypeScript 常量；（3）注册表声明表结构、运行时按路径加载。本项目用第（3）种：tables.registry.json 声明表名与文件，ConfigBootstrap.ensureGameConfigLoaded() 启动时加载，initData 将行挂到 Data.Character、Data.Skill 等命名空间。

技能语义拆为 Skill 与多行 SkillEffect：每行对应子弹、穿透/分裂/连锁修饰或直伤之一，EffectExecutor 按序解释——修饰器须在后续 bullet 效果之前，形成链式组合。伤害字符串（如 "atk*1.5+10"）由 FormulaParser 在白名单运算下解

析，禁止 eval，以防配表篡改时代码注入。Targeting 支持自动索敌与简单指向输入。

相对可视化脚本，JSON 配表便于 Git 管理与 diff；部分错误会推迟到集成阶段，因此以 isGameConfigReady 与单元脚本（公式、属性修饰器）作质量门禁。

2.2.4 Roguelite 跑图与关卡结构

元游戏采用 Slay the Spire 式 DAG 跑图：每局三个 Act，每 Act 为多层有向无环图，节点含休息、普通战斗、Boss、宝藏等^[7]。地图生成一般含分层、节点抽样、连边约束与可达性校验；RunMapGenerator 实现固定行数网格、末行 Boss、中间随机分叉，validateActReachBoss 保证 Boss 可达。失败时回退保底图，避免进度卡死。

SeededRng 使相同种子可复现地图，便于答辩录像与回归。MetaFlowController 在用户确认节点后触发 onEnterCombat，将节点载荷（如 Boss 关更长生存时间）传入战斗场景，元游戏与局内逻辑松耦合。

2.2.5 战斗性能与密集实体仿真

Survivor-like 后期同屏怪物与弹幕增多，H5 热点包括：每帧大量碰撞、预制体反复实例化触发 GC、主线程追逐与分离计算。常见对策有对象池、纹理合批、逻辑暂停、纯数值计算迁至 Web Worker^[7, 7]。

BulletPool、MonsterPool 按预制体路径分桶复用节点；BulletHitTest 用距离平方减少开方；GameSession.paused 在技能面板打开时让战斗 System 早退。实体数达阈值时，CombatDataPrepareSystem 将坐标等快照经 CombatDataBridge 提交 Worker，在共享的 combatWorkerLogic.ts 中计算追逐、分离与摆动，主线程再写回速度；子弹碰撞依赖 Laya 节点与阵营判定，仍留主线程。Cantrell 等在 Unity 做物理人群疏散时同样强调重计算与交互渲染分离^[7]；Helbing 等关于恐慌人群的研究提示密集场景须关注分离力与稳定性^[7]，与 MonsterChaseSystem 中分离项设计一致。

上述选型在可扩展与可测之间折中：ECS 与配表支撑内容迭代，池化与 Worker 支撑同屏规模，跑图 DAG 支撑元游戏节奏。

2.3 UI 架构选择

2.3.1 MVC 分层与全屏界面栈

游戏 UI 常见模式有 MVC、MVP、MVVM 等^[7]。Heijstek 等比较架构描述的图文形式，指出结构化图示有助于理解系统边界；本项目采用 **MVC** 变体：

- **Model**: 来自 ECS 组件(如 hp、装配状态)或独立模型类(如 `RunMapPanelModel`、`RunMapState`)；
- **View**: Laya 预制体与 UI2 节点，负责显示与布局；
- **Controller**: 继承 `UiControllerBase`，处理生命周期、输入与路由，不直接操作战斗场景实体节点。

元游戏含开始、选关、跑图等多个全屏界面。多个面板同时 `visible` 时易出现遮挡与事件穿透。`UIStackManager` 维护路由栈：`push(routeId)` 入栈并显示对应 Controller，`pop()` 出栈恢复上一层，保证任意时刻仅一个全屏界面占焦点。该做法与 Gregory 所述界面状态机与玩法逻辑解耦一致^[7]。

2.3.2 战斗内 UI: HUD、暂停与技能装配

局内 UI 含主 HUD（血条、经验）、升级奖励面板、死亡重启面板与 **Tab** 技能装配面板。`SkillSelectPanelController` 监听 Tab：打开时 `GameSession.paused` 为真，战斗 System 在 `update` 入口早退，怪物与子弹逻辑冻结，UI 仍接收指针事件——即输入层与战斗逻辑层分离，避免面板打开仍被攻击。

技能装配用长按拖拽：`SkillDragService` 按下超过阈值后创建顶层代理图标，规避 GBox 裁剪导致的命中错误；`SkillSlotHitTest` 统一坐标系判断落点属于装备栏或效果槽。关闭面板时，`SkillLoadoutSyncSystem` 将 `SkillLoadoutState` 原子写回 `Skill` 与 `PlayerAutoCastSystem`，避免半状态仍施放旧技能。图标由配表 `iconPath` 与 `SkillIconBinder` 加载，资源目录约定文件名与 id 对应。

跑图界面 `RunMapPanelView` 按 DAG 状态渲染节点与连线，`MapNodeIconUtil`、`LineLayoutUtil` 负责布局；玩家只能选与当前节点相邻且已解锁的边，规则由模型层校验，不在 View 硬编码。

2.3.3 UI 架构选型的综合评价

UI 架构选型归纳见表 2-3。界面不持有玩法真相，玩法状态以 ECS 与配表为准，UI 经 Controller 与 SyncSystem 读写；全屏流程用栈管理，局内装配用暂停位切断战斗 tick。该划分降低循环依赖，也使第 7 章功能测试（Tab 暂停、装配同步、跑图可达）有明确验收口径。

表 2-3 UI 架构选型要点汇总

问题	选型	理由
全屏导航	UIStackManager 路由	避免多面板同显
局内构筑	栈	
数据绑定	Tab + 拖拽 + Sync-System	暂停战斗、原子同步
与战斗边界	预制体 View + Controller	便于美术替换与动画
	禁止 System 操作 UI 节点	单向依赖、利于测试

2.4 本章小结

本章从引擎语言、玩法架构、UI 架构三方面说明 LayaAir + TypeScript、轻量 ECS + 配表 + Roguelite 跑图、MVC + UI 栈的技术路线及取舍，并明确不引入与周期不匹配的完整 DOTS 等方案。结论将在第 3 章转为可验证需求，在第 4 章落实为 ECS/MVC/Worker 设计，在第 5-7 章经实现与测试验证[?, ?, ?, ?]。

第 3 章 系统需求分析

本章在第 2 章技术选型基础上，给出 Survivor And Fight 的可行性论证、功能与非功能需求及验收口径，为第 4 章总体设计与第 7 章测试提供依据。需求描述以可验证为原则：每条 Must 级需求均可在代码或测试用例中找到对应实现或断言。

3.1 可行性分析

3.1.1 技术可行性

项目使用 LayaAir 3.x 与 TypeScript，已实现轻量 ECS、配表战斗、元菜单、DAG 跑图、技能装配 UI 及对象池/Worker 优化。引擎提供 2D 预制体、frameLoop 与输入封装；Chromium 内核浏览器支持 WebGL2 与 Web Worker。SyDRA 等研究表明理解引擎子系统边界有助于控制演化成本^[7]；Cantrell 等验证了在商业引擎上叠加领域逻辑（人群/怪物运动）的可行性^[7]。

已具备能力:ConfigBootstrap.ensureGameConfigLoaded 双通道加载配表;isGameConfigReady 作为战斗入口闸门；单元脚本覆盖 ECS 核心、公式解析与属性修饰器。

主要风险：同屏千怪帧率、Worker 与主线程逻辑一致性、UI2 拖拽命中——第 7 章五波实验与 T-F-05/06 等用例已部分缓解。

结论：在毕业设计周期内可交付可运行原型并完成论文，技术路线可行。

3.1.2 经济与社会可行性

作为教学原型，主要成本为开发与测试人力，无服务器与支付。游戏为奇幻战斗题材，须控制暴力表现并提示合理游戏时长；跑图休息节点提供节奏缓冲。不采集用户敏感信息，符合单机演示定位。

3.2 用户角色与运行环境

3.2.1 用户角色

系统仅设玩家单一角色：浏览主菜单、选关或跑图、局内移动与构筑、死亡后重启。无管理员后台；未来若扩展存档，仍保持“玩家”为唯一角色。

3.2.2 运行环境

表 3-1 运行环境建议配置

类别	要求
处理器	四核 2.0 GHz 及以上
内存	8 GB 及以上
显卡	支持 WebGL2
浏览器	Chrome / Edge 最新稳定版（答辩演示以 IDE 内置 Chromium 为准）
开发环境	LayaAir IDE 3.x、Node.js（工具脚本）、TypeScript

3.3 功能性需求

表 3-2 汇总主要功能模块；详细条目见第 3.4 节。

表 3-2 主要功能性需求一览

模块	需求描述
ECS 核心	创建/销毁实体；按类型存取组件；分组更新
玩法实体	玩家/怪物标签；移动、视图同步；hp/maxHp 与 modifier
技能战斗	多 effect 技能；自动施法；子弹表驱动伤害与穿透/分裂/连锁
怪物系统	波次刷怪；追逐与分离；接触伤害；离屏回收
进度系统	经验、升级、随机奖励
元游戏	开始/选关；onEnterCombat；生存计时胜利
跑图	三幕 DAG；节点类型；种子可复现；Boss 可达
技能装配 UI	Tab 暂停；三装备栏；长按拖拽装配
配置	JSON 注册加载；缺失时 DEFAULT_* 与去重日志

3.3.1 用例：局内战斗

参与者：玩家。前置条件：`isGameConfigReady()==true`，`SimpleEcsDemo.init()` 完成。主成功场景：（1）输入移动；（2）`PlayerAutoCastSystem` 自动施法；（3）子弹碰撞并应用穿透/分裂/连锁；（4）击杀获经验并升级；（5）Tab 打开技能面板，`GameSession.paused=true`，装配后恢复。扩展：`hp≤0` 时显示重启面板。对应第 5.5 节场景一、二。

3.3.2 用例：元游戏进入战斗

用户在跑图或选关界面确认节点,MetaFlowController 触发 onEnterCombat(payload); Main 显示战斗容器并 demo.init(), CombatSurvivalTimer 启动; Boss 节点载荷可携带更长胜利时间。

3.4 功能需求详细说明

3.4.1 ECS 核心

须提供实体创建/销毁 API, 销毁时移除全部组件; 同类型组件不可重复挂载; 系统按 input/logic/render 分组与优先级确定性调度; 支持按组件类型遍历及命名筛选器查询^[?]。

3.4.2 战斗与技能

玩家须具备 hp/maxHp 与可溯源 modifier。技能由配表定义冷却与 effect 列表; 至少三个装备栏独立冷却。效果类型覆盖 bullet、modifier_*、direct_damage; 子弹带阵营, 仅对敌对阵营生效; 连锁半径有界, 避免无界遍历^[?]。

3.4.3 怪物、元游戏与 UI

怪物在玩家周围环带生成并保持最小间距; 须实现追逐、接触伤害、离屏回收; 击杀给予经验 (含等级加成)。跑图每局三幕 DAG, 仅沿已解锁邻接边前进; validateActReachBoss 保证可达。全屏界面经 UIStackManager; Tab 暂停战斗; 拖拽区分技能/效果槽; 血条与升级面板实时反馈^[?]。

3.5 非功能性需求

性能: 目标稳定 60 FPS; 同屏规模由 MONSTER_COUNT 与波次控制; 对象池与 Worker 按第 4.2.3 节启用。

可维护性: 模块边界与第 4 章一致; 静态常量集中于 defines。

可靠性: Worker/配表/跑图失败时的回退路径见第 4.4 节。

兼容性: 优先 Chromium; Worker 不可用时 computeSync 主线程回退^[?]。

表 3-3 非功能需求量化指标（验收参考）

指标	目标值	测量方法
战斗帧率（常规波次）	均值 ≥ 55 FPS	TestLevelFpsTracker
千怪消融 P95	池化后显著低于无池	表 7-6
配表就绪	Data.Skill 非空	isGameConfigReady
Worker 行为	与主线程无逻辑漂移	轨迹对比（见第 7 章）

3.6 数据、接口与约束

逻辑数据实体：运行时 ECS 组件；配表行（Character、Skill、SkillEffect、Bullet）；RunMapState 图结构；GameSession、SkillLoadoutState。

关键接口：(1)ensureGameConfigLoaded(): Promise<boolean>;(2)onEnterCombat(payload: ControlInputSource);(4)UIStackManager.push/pop;(5)CombatWorkerComputeRequest/Response 字段版本兼容。

约束与假设：浏览器启用 JavaScript 与 WebGL；开发阶段已执行 sync-config-to-bin.ps1；单机本地运行；2D 俯视角；同屏数百实体级，非 MMO 规模。

3.7 需求优先级

表 3-4 需求优先级（MoSCoW）

级别	需求
Must	ECS 战斗、子弹碰撞、移动、刷怪、配表、主菜单进战斗、生存胜利
Should	跑图 DAG、技能装配、升级奖励、对象池与 Worker
Could	法杖三槽完整 UI、全量 Effect 独立特效
Won't	联机、账号、支付（本期）

3.8 需求与测试映射

表 3-5 列出需求与验证手段的对应关系，供第 7 章追溯^[7]。

3.9 本章小结

本章论证技术与社会可行性，明确用户环境、功能/非功能需求、接口约束及 MoSCoW 优先级，并建立需求—测试映射。下一章给出分层架构、ECS/MVC/Worker 设计与可靠性策略。

表 3-5 需求与测试映射（节选）

需求	验证方式	编号/脚本
ECS 实体销毁 清理组件	单元验证	ecsCorePhase1.verify.ts
Tab 暂停与装 配同步	手工用例	T-F-05、T-F-06
Boss 可达 公式安全解析	生成断言 单元验证	validateActReachBoss formulaParser.verify.ts
对象池降 P95	性能消融	表 7-6 第 4-5 波

第 4 章 系统总体设计

4.1 设计原则

系统总体设计遵循以下原则，并在后续模块实现中反复体现：

- (1) 数据与逻辑分离：组件仅存数据，系统执逻辑；可调参数尽量外置 JSON 配表，工程常量集中于 `src/defines/`^[?]。
- (2) 单一职责：`BulletSystem` 负责子弹运动与碰撞；`EffectExecutor` 负责配表效果解释；`MetaFlowController` 仅协调元游戏流程，不直接改写 ECS 组件^[?]。
- (3) 可验证：核心模块提供公式解析、ECS 核心、属性修饰器等单元验证脚本，变更后可快速回归^[?]。
- (4) 渐进复杂度：第 1 期采用轻量 ECS 与单线程调度；对象池与 Web Worker 仅在实体规模达标时启用。
- (5) 低耦合：UI 经 Controller 与 `SyncSystem` 读写 ECS；战斗 Worker 与主线程共享 `combatWorkerLogic.ts`，避免双份逻辑漂移^[?]。
- (6) 可降级：配表缺失、Worker 不可用、跑图生成失败时均有明确回退路径（见第 4.4 节与第 4.5 节）。

4.2 系统具体设计

Survivor And Fight 在逻辑上划分为表现层、UI 控制层、游戏逻辑层、领域服务层与基础设施层，如图 4-1 所示。`Main.ts` 作为应用入口，在 `Laya.timer.frameLoop` 中驱动 `EcsWorld.update`，再由引擎完成渲染与 UI 事件分发。元游戏（菜单、选关、跑图）与局内战斗共用同一 TypeScript 工程，通过容器显隐与 `MetaFlowController` 回调切换，避免维护两套入口。

4.2.1 基于 ECS 的 Gameplay 实现

实体—组件—系统是局内玩法的核心范式。`EntityId` 为数值标识；`ComponentStore` 按组件类型使用 `Map<EntityId, Component>` 存储；`EcsWorld` 按 `input/logic/render`

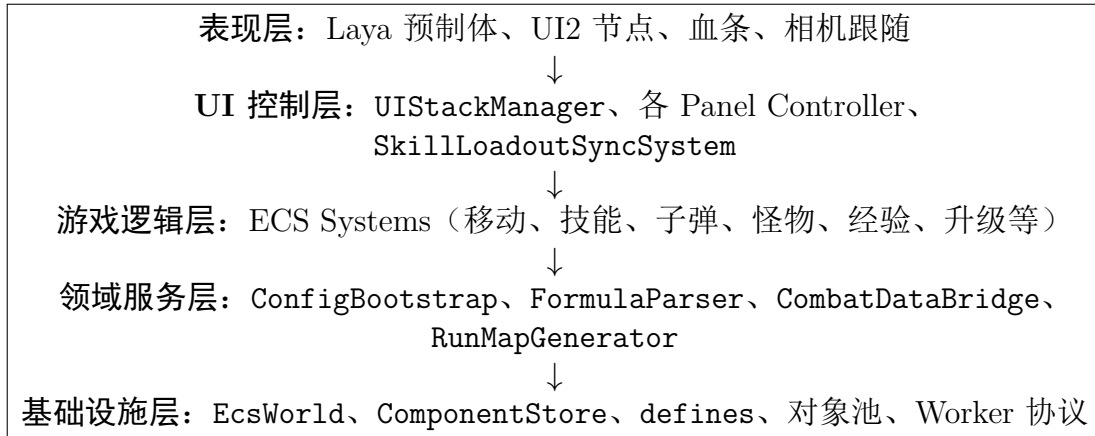


图 4-1 系统分层架构

分组顺序调用已注册系统。玩法数据与逻辑严格分离:Position、Velocity、Attribute、Skill、SkillLoadoutState 等仅存字段;MovementSystem、SkillSystem、BulletSystem 等无状态遍历实体并修改组件。

标签与筛选:玩家与怪物共享位移、属性、技能等组件,仅通过 PlayerTag/MonsterTag 区分。FilterRegistry 提供 Players、Monsters 等命名筛选器,避免各 System 重复编写交集查询。该设计体现组合优于继承:新增怪物行为时扩展组件字段或新增 System,而非维护 MonsterBase 类树。

战斗数据流:每帧 CombatDataPrepareSystem 采集坐标等快照 → 可选 CombatDataBridge 提交 Worker 执行 processCombatFrame → 主线程 MonsterChaseSystem 应用追逐/分离结果 → BulletSystem 在主线程完成碰撞(依赖 Laya 节点与阵营判定)。GameSession.paused 在死亡、技能 Tab 或胜利时冻结逻辑 System 的 update 入口。

配表驱动:Skill 与多行 SkillEffect 描述冷却与效果链;EffectExecutor 按序解释 bullet、modifier_*、direct_damage。修饰器须位于后续 bullet 之前,形成链式组合语义。伤害字符串由 FormulaParser 在白名单运算下解析,禁止 eval。

4.2.2 基于 MVC 的 UI 结构实现

元游戏与局内 UI 采用 MVC 变体^[7]:

- **Model:** ECS 组件(hp、装配状态)或独立模型(RunMapPanelModel、RunMapState);
- **View:** Laya 预制体与 UI2 节点绑定,负责布局与显示;
- **Controller:** 继承 UiControllerBase,处理生命周期、输入与路由,禁止直接操作战斗场景中的实体节点。

全屏导航: `UIStackManager` 维护路由栈, `push(routeId)` 入栈并显示对应 `Controller`, `pop()` 恢复上一层, 保证任意时刻仅一个全屏界面占据焦点。开始、选关、跑图等界面经 `MetaMenuBootstrap` 注册路由。

局内交互: `SkillSelectPanelController` 监听 `Tab`, 打开时将 `GameSession.paused` 置真, 战斗 `System` 早退而 `UI` 仍接收指针事件。 `SkillDragService` 长按拖拽, `SkillSlotHitTest` 统一命中判定; 关闭面板时 `SkillLoadoutSyncSystem` 原子写回 `Skill` 与 `PlayerAutoCastSystem`。 `RunMapPanelView` 根据 `DAG` 状态渲染节点与连线, 可达性规则由 `Model` 校验而非 `View` 硬编码。

核心约束: 界面不拥有玩法真相——玩法状态以 `ECS` 与配表为准, `UI` 仅经 `Controller/SyncSystem` 读写。

4.2.3 基于 Web Worker 和对象池的优化实现

对象池: `BulletPool`、`MonsterPool` 按预制体路径分桶复用 `Laya` 节点, 抑制 `instantiate/destroy` 触发的 `GC` 尖峰。 `MonsterRecycleSystem` 在怪物远离玩家超过阈值后回收实体, 维持活跃集合规模。

Web Worker: 当场上实体数 $\geq \text{COMBAT_WORKER_MIN_ENTITY_COUNT}$ 时, `CombatDataPrepareSy` 将快照经 `CombatDataBridge` 发送至 `Worker`; `combatWorkerLogic.ts` 在主线程与 `Worker` 间共享, 计算追逐、分离与摆动。 `CombatDataBridge.prepareFrame` 优先消费异步结果, 失败或未就绪时调用 `computeSync` 回退。子弹碰撞、连锁与分裂仍保留主线程, 因依赖引擎节点与即时命中判定。

逻辑暂停: 除 `Worker/池化` 外, `GameSession.paused` 在技能面板、死亡与胜利结算时切断战斗 `tick`, 属于零成本“冻结”手段, 与 `Cantrell` 等强调的“重计算与交互渲染分离”思路一致^[7]。

4.3 系统交互与时序

图 4-2 与图 4-3 给出从应用启动到局内战斗、元游戏切换与重启的端到端交互时序。图源为 `figures/system-sequence.mmd`, 经 `Mermaid CLI` 导出为 `PNG` 嵌入 `PDF`; 下图为文字版摘要, 便于无图环境阅读。

4.4 可靠性设计

可靠性设计目标是在配表异常、`Worker` 不可用或地图生成失败时, 仍可进入可玩或可调试状态, 而非直接崩溃。实现要点如下。

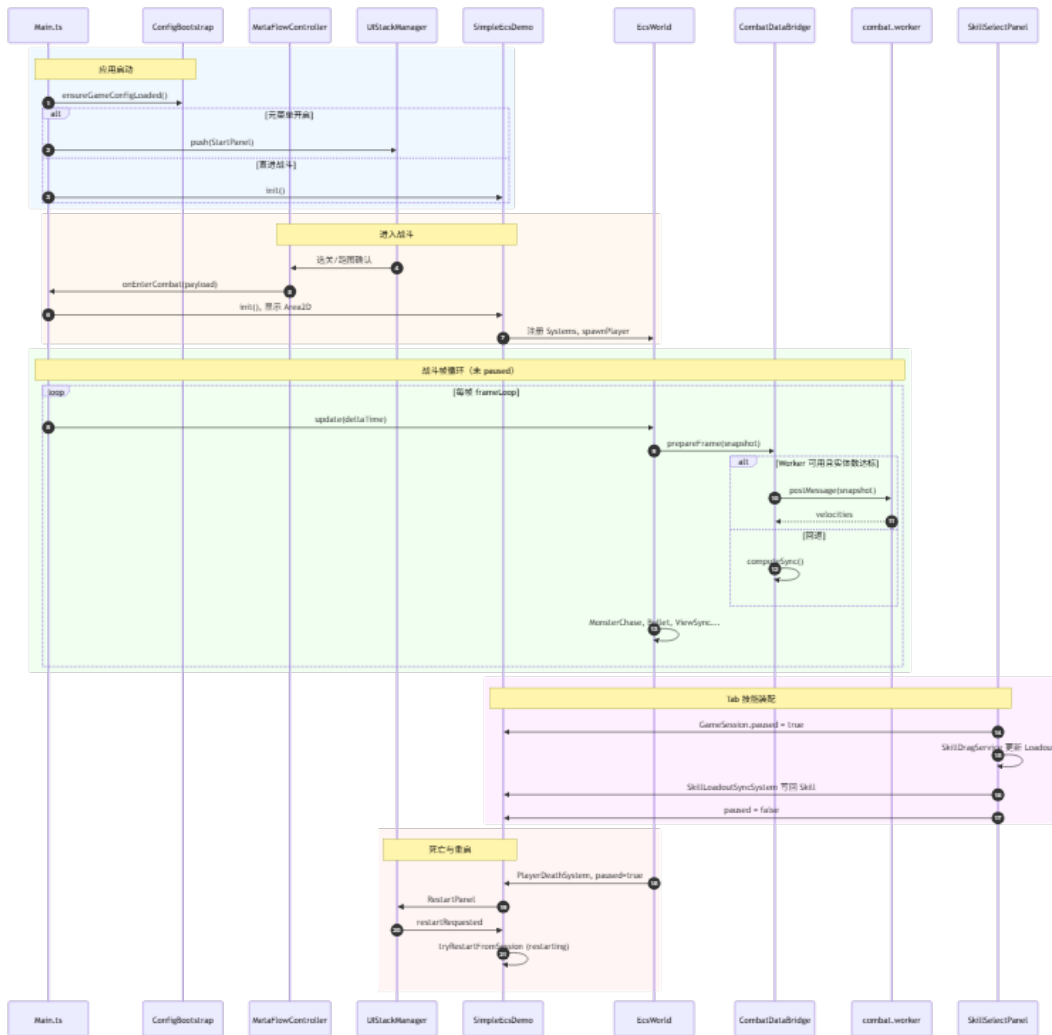


图 4-2 系统主要交互时序（Mermaid 序列图）

- (1) 应用启动 Main → ConfigBootstrap: `ensureGameConfigLoaded`
→ MetaMenuBootstrap: `UIStackManager.push(Start)` 或 `SimpleEcsDemo.init`
- (2) 进入战斗 `RunMapPanel/SelectLevel` → `MetaFlowController.onEnterCombat`
→ Main: 显示 `Area2D`, `demo.init()`, 启动生存计时
- (3) 战斗帧循环 `frameLoop` → `EcsWorld.update` (若未 `paused`)
→ `CombatDataPrepareSystem` → `CombatDataBridge` (Worker/同步)
→ `MonsterChaseSystem` → `BulletSystem` → `ViewSyncSystem`
- (4) 施法链 `PlayerAutoCastSystem` → `CastPlan` → `EffectExecutor`
→ `BulletSystem.spawn` → `AttributeSystem` (伤害)
- (5) 升级 `MonsterDeath` → `ExperienceSystem` → `UpgradeRewardSystem`
→ `RewardPanel` → `RewardApplyService` → `AttributeSystem`
- (6) Tab 装配 `SkillSelectPanelController: paused=true`
→ `SkillDragService` → `SkillLoadoutSyncSystem` → `paused=false`
- (7) 死亡重启 `PlayerDeathSystem: paused=true` → `RestartPanelSystem`
→ `SimpleEcsDemo.tryRestartFromSession` (`restarting` 防重入)

图 4-3 系统主要交互时序（文字摘要）

(1) 配表加载与就绪判据。`ConfigBootstrap.ensureGameConfigLoaded` 遍历 `configBases()`, 依次尝试 `fetch` 与 `Laya.loader` 双通道加载 `tables.registry.json` 及各表; `isGameConfigReady()` 要求 `configLoaded` 且 `Data.Skill` 非空, 作为“可否进入战斗”的全局闸门。`Main.onStart` 与 `SimpleEcsDemo.spawnPlayer` 均在生成实体前等待或复查该状态。

(2) 运行时缺表降级。`SimpleEcsDemo` 维护 `missingConfigLogged: Set<string>`, 通过 `logMissingConfigOnce` 对每个缺失键仅告警一次, 避免刷屏。`defines` 中 `DEFAULT_PLAYER_HP` `DEFAULT_PLAYER_PREFAB` `DEFAULT_PLAYER_AUTO_SKILL_ID` 等在查表失败时回退; 升级奖池表为空时返回内联默认条目, 保证演示不中断。

(3) **Worker** 双路径。`CombatDataBridge.initWorker` 失败、`onerror` 或 `postMessage` 异常时将 `workerUsable=false`; `prepareFrame` 无可用异步结果时调用 `computeSync`, 与 `Worker` 共用 `processCombatFrame`。`MonsterChaseSystem` 无 `Worker` 结果时走本地追逐 + 分离; `BulletSystem` 在存在连锁弹等 `Worker` 未覆盖场景时回退完整主线程逻辑。实体数低于 `COMBAT_WORKER_MIN_ENTITY_COUNT` 时不提交 `Worker`, 减少通信开销。

(4) 跑图生成保底。`RunMapGenerator.generate` 在最多 `RUN_MAP_GENERATION_MAX_RETRIES` 次随机尝试后, 若 `validateActReachBoss` 仍失败, 则调用 `generateFallback` 生成三幕各「休息 → 战斗 → Boss」线性图, 保证 Boss 可达、进度不卡死。

(5) 重启防重入。`RestartPanelSystem` 将 `session.restartRequested` 置真; `SimpleEcsDemo.tryRestartFromSession` 在 `restarting=true` 期间异步清场与重生, `update` 中避免重复触发, `finally` 复位标志, 防止监听器与实体重复注册。

(6) 预制体加载失败。实例化失败时 `createPlaceholderNode()` 生成占位碰撞体, 便于在无美术资源环境下继续调试逻辑。

4.5 容错与降级策略

表 4-1 汇总主要故障模式与系统响应, 与第 4.4 节实现一一对应。

暂停语义统一: `GameSession.paused` 同时服务死亡暂停与 Tab 装配暂停; `RestartPanelSystem` 通过 `isPlayerDead()` 区分是否弹出重启面板, 避免技能面板误触重启流程。

4.6 可扩展性设计

本系统的可扩展性建立在 ECS 的组合优于继承原则之上, 而非为每种怪物或技能派生子类。

表 4-1 容错与降级策略

故障模式	检测方式	降级行为
配表 JSON 404 或空表	<code>isGameConfigReady()=false</code>	控制台错误提示；玩家/技能使用 <code>DEFAULT_*</code> ；自动施法可能退化
Worker 初始 化/通信失败	<code>workerUsable=false</code>	<code>computeSync</code> 主线程重算追逐
实体数低于 Worker 阈值	快照计数	不投递 Worker，全主线程
跑图随机生成不可达	<code>validateActReachBoss</code>	<code>generateFallback</code> 线性保底图
预制体路径无效	<code>instantiate</code> 异常	占位节点 + 日志
UI 路由加载失败	Controller 构造异常	跳过该路由并日志，不阻断战斗容器

实体扩展：新玩法维度优先新增组件类型（如未来的 `BuffComponent`、`ShieldComponent`），再在独立 System 中读写；已有 `MonsterTag` 实体自动参与 `FilterRegistry.Monsters`，无需修改 `EcsWorld` 内核。

行为扩展：新行为对应新 System 或扩展现有 System 的分支；例如新增“光环”只需注册 `AuraSystem` 并在 `SimpleEcsDemo` 构造函数中 `addSystem`，不必改动 `MovementSystem`。

技能扩展：新 `effect` 类型在 `EffectExecutor` 增加分支；修饰器仍通过“写上下文 → 后续 bullet 消费”扩展，策划在 `SkillEffect` 表调序即可组合。新子弹形态仅需配表 `Bullet` 行与预制体路径。

内容扩展：新跑图节点类型扩展 `RunNodeType` 与 `RunMapGenerator` 权重表；新全屏界面在 `UIStackManager` 注册路由与 Controller，不修改战斗 ECS。

与继承方案对比：若采用 `Player extends Character extends Actor`，每增加一种“会分裂的精英怪”都需新建子类并重写 `update`，导致 Laya 节点与逻辑深度耦合。当前方案下，精英怪仅是 `MonsterTag` + 额外组件（如更高 `Attribute` 或专用 `MonsterDef.waveWeight`），行为差异由 `MonsterAutoCastSystem` 等组合完成，符合 Gregory 与 ECS 文献对“数据导向、增量扩展”的论述^[7]。

4.7 本章小结

本章给出设计原则、分层架构，并从 ECS Gameplay、MVC UI、Worker/对象池三方面说明系统具体设计；进一步给出端到端时序、可靠性机制、容错表与基于组合的可扩展性策略，为第 5 章实现细节提供结构框架。

第 5 章 核心模块详细设计与实现

本章按第 4 章总体设计，结合 `src/` 源码说明核心模块实现。`SimpleEcsDemo` 为组合根（Composition Root），集中注册系统、池与 UI 控制器，是阅读仓库的首选入口。

5.1 ECS 架构 Gameplay 实现

5.1.1 世界、组件与筛选器

`EntityManager` 使用自增 ID 与空闲列表复用实体。`ComponentStore` 按类型维护 `Map<EntityId, Component>`。`EcsWorld.update` 按 `input/logic/render` 分组顺序调用系统^[7]。`FilterRegistry` 在构造时 `registerNamedFilters`，提供 `Players`、`Monsters` 等命名筛选。

5.1.2 属性、移动与自动施法

`AttributeSystem` 支持按来源增删 `modifier`，供 `Buff` 与升级奖励使用。`MovementSystem`、`ControlSystem` 处理位移与输入。`PlayerAutoCastSystem` 对三装备栏维护独立冷却与 `pendingCasts`；`MonsterAutoCastSystem` 驱动怪物施法。

5.1.3 技能与效果执行链

`PlayerAutoCastSystem` 决定施放后，流程为：（1）从 `SkillLoadoutState` 读取技能 id 并检查 `Skill.cooldownRemain`；（2）`CastPlan` 按配表顺序解析 `effect` 行；（3）`EffectExecutor` 分发：`modifier_*` 写入分裂/连锁/穿透上下文，供下一条 `bullet` 消费；（4）`bullet` 调用 `BulletSystem.spawnBulletWithOptions`；（5）`direct_damage` 经 `FormulaParser` 扣血。

表 5-1 效果类型与执行器映射

effect 字段	行为
<code>bullet</code>	<code>BulletSystem.spawnBulletWithOptions</code>
<code>modifier_split</code>	设置分裂数量
<code>modifier_chain</code>	设置连锁与半径
<code>modifier_pierce</code>	增加穿透
<code>direct_damage</code>	公式直伤

5.1.4 子弹碰撞与怪物 AI

BulletSystem 从 BulletPool 取节点, 用 hitRadiusSq 做圆碰撞; 命中后扣血、递减穿透, 并按配置处理连锁与分裂^[7]。MonsterChaseSystem 计算追逐方向, 叠加分离力与随机摆动; 有 Worker 结果时 applyWorkerVelocities, 否则本地计算。MonsterWaveSpawnSystem 在环带刷怪; MonsterRecycleSystem 与 MonsterPool 回收离屏单位。MonsterContactDamageSystem 处理接触伤害。

5.1.5 经验、升级与元游戏数据

ExperienceSystem 在击杀时增加经验并判断升级。UpgradeRewardSystem 打开奖励面板; RewardPoolService 按权重抽取条目, RewardApplyService 写回属性或技能。RunMapGenerator 生成三幕 DAG, 失败时 generateFallback。MetaFlowController 协调 onEnterCombat 并将节点载荷传入战斗场景。

5.2 MVC 架构 UI 实现

5.2.1 UI 栈与元游戏界面

MetaMenuBootstrap 向 UIStackManager 注册 StartPanel、SelectLevelPanel、RunMapPanel 等路由。用户确认节点后触发 onEnterCombat, Main 隐藏菜单容器、显示战斗 Area2D 并 demo.init()。RunMapPanelView 使用 MapNodeIconUtil、LineLayoutUtil 渲染 DAG; 仅允许选择已解锁邻接边。

5.2.2 战斗 HUD 与技能装配

MainHudSystem、BloodBarSyncSystem 将 ECS 中 hp 映射至 ProgressBar。SkillSelectPanelController 绑定 MainUIPanel.lh, Tab 切换时设置 GameSession.paused。SkillDragService 在超过 LONG_PRESS_MS 后使用顶层代理图标拖拽; SkillSlotHitTest 判断装备栏/效果槽落点。SkillLoadoutSyncSystem 关闭面板时原子写回 Skill 与 PlayerAutoCastSystem, 并调用 getCombatEffectIds 刷新可战斗 effect 集合。

5.2.3 输入与重启面板

InputService 封装键盘/指针; ControlInputAdapter 转为 ControlSystem 可读方向。Tab 打开面板时战斗 Control 不更新, UI 仍可接收事件^[7]。PlayerDeathSystem

在 `hp ≤ 0` 时置 `paused; RestartPanelSystem` 经 UI 栈显示重启面板; `tryRestartFromSession` 在 `restarting` 保护下清场重生。

依赖关系保持单向: `UI → ECS; Systems → 配表`; 禁止 `System` 直接操作 UI 节点。

5.3 Web Worker 和对象池的优化实现

`BulletPool`、`MonsterPool` 按预制体路径分桶, `acquire` 优先复用空闲节点, 不足时再 `instantiate`。 `BulletHitTest` 使用距离平方减少开方。

`CombatDataPrepareSystem` 在 `isPaused()` 为假时打包怪物坐标等快照, 调用 `CombatDataBridge.prepareFrame`。桥接层在 Worker 就绪且实体数达标时 `postMessage`, 否则 `computeSync`; `MonsterChaseSystem` 与 `BulletSystem` 按是否有 Worker 结果及是否存在连锁弹, 选择应用速度或走完整主线程碰撞。性能消融可通过 `MetaRunSession.testUseCo` 与 `testDefine` 预设控制 (见第 6 章)。

5.4 游戏主循环设计

5.4.1 入口与配置时序

`Main.onStart` 创建 `InputService`、`ControlInputAdapter`、`SimpleEcsDemo`, 注册 `frameLoop`, 并 `await ensureGameConfigLoaded()`。配表优先于实体生成, 避免 `Data.Character.GetByID` 为空。若 `META_MENU_ENABLED` 为真则 `startMetaMenu`, 否则直接 `demo.init()`。

5.4.2 SimpleEcsDemo 构造与 init

构造函数内: 创建 `FilterRegistry`; 注册 `AttributeSystem`、`ExperienceSystem`、`BulletSystem` (注入池与桥接)、`CombatDataPrepareSystem`、`SkillSystem`、`MonsterWaveSpawnSystem`、`UpgradeRewardSystem`、`MainHudSystem`、`SkillSelectPanelController` 等。 `init()` 在配表就绪后按 `Character` 行加载玩家预制体, 挂载 `PlayerTag`、`Position`、`Attribute`、`Skill`、`SkillLoadoutState`、`ViewComponent` 等; 怪物由波次系统动态生成。

5.4.3 帧循环与暂停

每帧: 若 `restarting` 则跳过重启检测; 否则 `tryRestartFromSession`; 若 `isPaused()` 则战斗 `System` 早退; 否则 `world.update(deltaTime)`。生存计时与 Tab 暂停在 `Main`

层联动，ECS 层以 `GameSession.paused` 为唯一逻辑冻结开关。

5.5 典型场景实现剖析

5.5.1 场景一：玩家首次进入战斗

步骤 1: `Main.onStart` → `ensureGameConfigLoaded`。步骤 2: 元菜单 `UIStackManager.push(StackItem)` 战斗容器暂隐藏。步骤 3: 选关/跑图 → `onEnterCombat` → `demo.init()`, `CombatSurvivalTimer` 启动。步骤 4: 首帧 `ControlSystem`、`PlayerAutoCastSystem`、`MonsterWaveSpawnSystem` 协同运行。对应测试 T-F-01、T-F-09。

5.5.2 场景二：Tab 打开技能面板并更换装备

Tab → `paused=true` → 长按拖拽 → `SkillLoadoutSyncSystem` 写回 → `paused=false`, 下一冷却周期生效新技能。对应 T-F-05、T-F-06。

5.5.3 场景三：怪物密集时的 Worker 卸载

实体数 ≥ 阈值时 `CombatDataPrepareSystem` 提交 Worker；主线程仍处理子弹碰撞。关闭 Worker 或降低上限后 P95 帧时间上升（见第 7 章），说明优化有效且边界清晰。

5.6 关键代码文件索引

表 5-2 核心源码文件索引

模块	路径
入口	<code>src/Main.ts</code>
ECS 世界	<code>src/ecs/core/World.ts</code>
组件存储	<code>src/ecs/core/ComponentStore.ts</code>
战斗 Demo	<code>src/game/demo/SimpleEcsDemo.ts</code>
子弹	<code>src/game/bullet/BulletSystem.ts</code>
子弹池	<code>src/game/bullet/BulletPool.ts</code>
效果执行	<code>src/game/skill/EffectExecutor.ts</code>
Worker 桥接	<code>src/game/combat/CombatDataBridge.ts</code>
Worker 逻辑	<code>src/game/combat/combatWorkerLogic.ts</code>

模块	路径
跑图生成	src/game/run/RunMapGenerator.ts
元流程	src/game/meta/MetaFlowController.ts
UI 栈	src/game/ui/mvc/UIStackManager.ts
技能面板	src/game/ui/skillselect/SkillSelectPanelController.ts
配表引导	src/config/ConfigBootstrap.ts
静态常量	src/defines/*.ts

5.7 功能模块与论文章节对照

表 5-3 功能模块与论文章节对照

功能模块	论文章节
ECS Gameplay(实体、技能、子弹、怪物)	第 5 章第 5.1 节; 第 4 章第 4.2.1 节
MVC UI(栈、跑图、技能装配)	第 5 章第 5.2 节; 第 4 章第 4.2.2 节
Web Worker 与对象池	第 5 章第 5.3 节; 第 6 章
主循环与入口	第 5 章第 5.4 节
可靠性、容错与可扩展性	第 4 章第 4.4–4.6 节
测试与性能实验	第 7 章

5.8 本章小结

本章按 ECS Gameplay、MVC UI、Worker/对象池、主循环四条主线说明了实现细节，并以三个典型场景串联配置—流程—战斗的贯通路径。实现中显式采用组合根（SimpleEcsDemo）、对象池、桥接（CombatDataBridge）、策略（Targeting）、状态（GameSession.paused）等模式，未引入过度抽象。

主要难点与对策包括：(1)配表未同步至 bin/config 导致 404——以 sync-config-to-bin.ps1 与 isGameConfigReady 应对；(2)UI2 拖拽命中——顶层代理与 SkillSlotHitTest；(3)Effect 链顺序——文档与样例配表约束；(4)Worker 与主线程逻辑一致——共享 combatWorkerLogic.ts 与 computeSync 回退。表 5-2、表 5-3 便于评阅对照源码。下一章专述性能优化与实测数据。

第 6 章 性能优化与工程实践

本章在第 4.2.3 节设计基础上,说明 Survivor And Fight 已落地的性能优化、实验方法与工程实践,并与第 7 章实测数据对照。优化遵循先测量、再启用:对象池与 Worker 均可开关、可回退,避免过早引入难以调试的并发。

6.1 性能目标与瓶颈

目标:1920×1080 预览、Chromium 内核下,常规战斗波次平均帧率接近 60 FPS;千怪压力下帧时间尖峰可控(P95 可观测改善)。

瓶颈:H5 生存战斗常见热点为——(1)大量实体追逐、分离与子弹碰撞;(2)预制体反复 `instantiate/destroy` 引发 GC;(3)主线程 Scripting 与渲染争用预算^[?, ?]。Cantrell 等的人群仿真与 Bott 等的引擎工具链实践表明,重计算与交互渲染分离是常用对策^[?, ?];本项目在 2D H5 上用池化、Worker 与逻辑暂停应对同类问题。

6.2 对象池

BulletPool、MonsterPool 按 `prefabPath` 分桶维护空闲队列:`acquire` 优先复用并重置节点,不足时再 `Laya.Prefab.instantiate;put` 在子弹命中耗尽或 `MonsterRecycleSystem` 离屏回收时调用。

机制收益:分配从“每帧脉冲式新建/销毁”变为“峰值后平稳复用”,抑制 GC 停顿,改善 P95 帧时间。第 7 章第 4-5 波同窗消融中,开池后 P95 由 76.50 ms 降至 48.60 ms (约降 36.5%),平均 FPS 变化有限,说明瓶颈在尖峰而非均值吞吐。

6.3 Web Worker 卸载

`MonsterChaseSystem` 的追逐、分离、摆动为纯数值计算。`CombatDataPrepareSystem` 在实体数 $\geq$ `COMBAT_WORKER_MIN_ENTITY_COUNT` 时打包快照, `CombatDataBridge` 提交 Worker 执行 `processCombatFrame`;主线程与 Worker 共享 `combatWorkerLogic.ts`, `prepareFrame` 失败时 `computeSync` 回退。

边界:子弹碰撞、连锁与分裂依赖 Laya 节点与阵营判定,保留在主线程(见第 5.1 节)。五波实验未含“池 + Worker、已预热”的第六段同窗对照;第 5 波相对第 4 波仅开池关 Worker,可确证对象池价值,Worker 净收益待补充同窗实验(见

第 7 章)。

6.4 渲染与逻辑侧其它优化

碰撞微优化: `BulletHitTest` 使用距离平方, 减少每帧开方。

逻辑暂停: `GameSession.paused` 使战斗 System 在 Tab 装配、死亡与胜利时早退, 减少无效 tick。

动态图集: 接入 `AtlasConfig.atlascfg` 与 `TextureAtlasService` 后, 千怪千弹波次平均 FPS 由 13.32 升至 15.10 (约 +13.4%), 见表 7-4。

配置: 策划参数启动时一次加载; 工程常量集中于 `defines`, 避免热路径读盘。

6.5 帧预算与内存

以 60 FPS 计, 单帧预算约 16.67 ms。经验分配: 输入与 UI 1–2 ms; ECS 逻辑 (含子弹) 6–8 ms; Worker 通信 ≤ 1 ms (异步); 余量留给引擎渲染^[7]。超出预算时优先启用池化/Worker 或降低同屏上限, 不宜在业务层强行降分辨率。

JavaScript 引擎对短生命周期对象敏感; 未池化时大量 `instantiate/destroy` 易触发 GC 尖峰。对象池将分配脉冲抚平, 与 Chrome Performance 中 Scripting/GC 时间轴改善方向一致。项目以 `TestLevelFpsTracker` 与可选 Performance 面板观测, `missingConfigLogged` 等去重日志避免调试刷屏。

6.6 性能实验设计

6.6.1 实验目的与变量

验证对象池、动态图集及 Worker 对 FPS 与 P95 帧时间的影响^[7, ?]。

自变量: (A) 池化开/关; (B) Worker 开/关 (实体数达标时); (C) 同屏怪物规模。

因变量: 平均 FPS、P95 帧时间 (消融段)。

控制变量: 浏览器、分辨率、Level3 五波脚本、关闭后台占用。

6.6.2 实验步骤

1. 发布或 IDE 预览, 打开 DevTools Performance (可选);
2. 进入 Level3: 5 波 $\times$ 10 s, 第 1–3 波规模 10/100/1000, 第 4–5 波千怪消融;

3. `TestLevelFpsTracker` 记录均值；第 4-5 波记录 P95；

4. 对照表 7-5、表 7-6 并文字解读，避免只贴表不分析。

6.6.3 优化自检清单

合并性能相关变更前建议自检：热路径是否大量 `new`；销毁前是否 `pool.put`；碰撞是否用平方距离；`paused` 是否阻止无关 `System`；`Worker payload` 是否纯数值；配表是否启动时一次加载。

6.7 绿色计算与兼容性

节能：降低平均 CPU 占用有利于终端续航——池化减轻分配器压力，暂停减少空转，合理 `MONSTER_COUNT` 避免无意义满屏。

兼容性：`Worker` 不可用时自动 `computeSync`；配表 `fetch/Laya.loader` 双通道；不支持 `Worker` 的浏览器仍可运行主流程。浏览器或引擎升级后须回归 `Worker` 脚本路径与 `bin/config` 发布目录。

6.8 本章小结

本章说明对象池、Web Worker、碰撞/暂停/图集等优化手段及帧预算思路，并给出实验设计与自检清单。实测表明：对象池主要改善 P95 帧时间；动态图集在千怪弹幕场景提高平均 FPS；`Worker` 同窗定量结论待补充实验。下一章给出完整测试用例、环境与数据解读。

第 7 章 系统测试与验证

本章对 Survivor And Fight 开展单元验证、功能回归与性能实验，验证第 3 章需求与第 4 章设计是否落地。测试策略遵循 Petty 等对仿真与交互系统 V&V 的要求^[7, 8]：核心契约用脚本验证，玩法闭环用手工用例，性能用可复现关卡与日志统计。

7.1 测试策略

单元验证：ECS 核心、公式解析、属性修饰器——变更相关模块后须重跑对应 `*.verify.ts`。

集成/系统测试：跑图可达性、Tab 装配同步、Worker 轨迹一致性——手工或脚本断言。

性能测试：Level3 五波脚本 + `TestLevelFpsTracker`，对比表 7-5、7-6。

回归清单：修改 `FormulaParser`、`ComponentStore`、`AttributeSystem`、配表、`CombatDataBridge` 后，按附录需求—测试矩阵或表 3-5 回归主路径。

7.2 单元验证

7.2.1 ECS 核心

`ecsCorePhase1.verify.ts` 覆盖：实体 ID 唯一；组件增删与 `GetComponent`；销毁后不可访问；多 System 按分组与 priority 顺序执行^[7]。

7.2.2 公式解析器

`formulaParser.verify.ts` 覆盖：四则运算、括号、属性别名；非法字符与边界公式（以实现为准的安全返回/抛错）。配表伤害依赖解析器，任何变更须 rerun。

7.2.3 属性修饰器

`attributeModifier.verify.ts` 覆盖：多 modifier 叠加；按 `sourceId` 批量移除；`getModifierContributions` 可追溯。升级奖励与 Buff 均依赖该机制^[7]。

7.3 功能测试

表 7-1 为代表性手工用例，覆盖战斗、元游戏、跑图、装配与重启主路径。

表 7-1 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动并等待加载	isGameConfigReady, 技能表非空
T-F-02	进入战斗移动	角色与相机正常
T-F-03	怪物接近	追逐、扣血、血条更新
T-F-04	观察自动攻击	子弹命中; 穿透/分裂/连锁
T-F-05	Tab 打开技能面板	paused=true, 可拖拽装配
T-F-06	关闭面板	战斗恢复, 下一冷却周期新技能生效
T-F-07	升级	奖励面板与应用
T-F-08	玩家死亡	重启面板, restarting 无重入
T-F-09	元游戏进战斗	生存计时启动
T-F-10	Boss 节点	Boss 胜利时长参数生效

7.3.1 集成测试补充

跑图可达性:对 RunMapGenerator.generate 多次随机种子断言 validateActReachBoss; 失败应走 generateFallback。

装配同步:互换装备栏技能后,下一冷却周期伤害/图标与装配一致,否则 SkillLoadoutSyncSystem 存在缺陷。

Worker 一致性: Worker 开/关各运行一段时间,怪物轨迹允许一帧延迟,但无逻辑漂移^[?]。

7.4 性能测试

7.4.1 测试环境

表 7-2 为五波实验与功能抽测的统一环境(2026-05-18, 单次运行)。

选关 Level3 共 5 波: 第 1-3 波怪物 10/100/1000, 装备三把测试法杖; 第 4-5 波均为 1000 怪, 依次为「无池无 Worker / 仅池(关 Worker)」, 用于同窗消融。第 3 波含首次千怪冷启动, 不纳入与第 4-5 波的公平对比。

表 7-2 测试环境配置

项目	配置
操作系统	Windows 10/11（64 位）
运行方式	LayaAir IDE 内置浏览器预览（Chromium 内核）
视口 / 设计分辨率	1920×1080 预览；设计稿 1334×750，showall 缩放
引擎与语言	LayaAir 3.x，TypeScript
测试关卡	选关 Level3：5 波 × 10s，双方无敌
统计工具	TestLevelFpsTracker（控制台日志）

7.4.2 动态图集对比（前三波）

表 7-3、7-4 为接入动态图集前后的历史两轮采样（仅前三波）。第 3 波（1000 怪）FPS 由 13.32 升至 15.10（约 +13.4%），说明重度弹幕下纹理合批有效。

表 7-3 测试关卡分波 FPS（接入动态图集，前三波）

波次	同屏怪物	单发子弹	平均 FPS	采样帧数
1/3	10	10	56.74	569
2/3	100	100	40.72	408
3/3	1000	1000	15.10	152
前三波汇总	—		37.48	1129

表 7-4 动态图集接入前后 FPS 对比（前三波）

波次（怪物数）	对照 FPS	动态图集 FPS	变化
1（10）	58.68	56.74	-1.94
2（100）	46.30	40.72	-5.58
3（1000）	13.32	15.10	+1.78
前三波汇总	39.46	37.48	-1.98

7.4.3 五波完整帧率

表 7-5 为同次运行、已接入图集与 DrawCall 合批的完整五波数据。整场 1733 帧、约 50.3s，平均 FPS 约 34.48。

7.4.4 对象池消融（第 4—5 波）

结果解读

1. 规模曲线（第 1—3 波）。怪物 10→100→1000 时 FPS 由 58.62 降至 15.14，符合实体与弹幕压力上升规律。

表 7-5 Level3 五波测试帧率（单次运行）

波次	怪物数	池	Worker	平均 FPS	P95 (ms)
1	10	开	开	58.62	—
2	100	开	开	46.84	—
3	1000	开	开	15.14	—
4	1000	关	关	26.16	76.50
5	1000	开	关	25.98	48.60
前五波汇总	—	—	—	34.48	—

表 7-6 性能对比实验（对象池消融，1000 怪 / 10s）

配置	平均 FPS	P95 (ms)	对应波次
基线（无池、无 Worker）	26.16	76.50	第 4 波
仅对象池（关 Worker）	25.98	48.60	第 5 波

参考：第 3 波（池 +Worker 全开，首次千怪）平均 FPS 15.14，不宜与上表直接对比。

2. 平均 **FPS**：池化对均值影响有限。第 4、5 波均值接近（26.16 对 25.98），预热后瓶颈主要在子弹碰撞与绘制。
3. **P95**：对象池显著平滑帧时间。P95 由 76.50 ms 降至 48.60 ms（约降 36.5%），印证第 6 章关于 GC/实例化尖峰的论述。
4. 第 3 波低于第 4 波基线。第 3 波含资源预热与首次千怪开销；消融结论以第 4-5 波为准。
5. **Web Worker**。本次未设「池 +Worker、已预热」同窗段；Worker 定量收益列为后续工作。

7.5 系统运行截图

插图位于 thesis/figures/，编译时若 PNG 存在则自动嵌入。图 7-1-7-6 覆盖元游戏与局内闭环。

7.6 测试结论

功能：Must 级需求经 T-F-01-10 与单元脚本覆盖；Should 级跑图、装配、升级、池化/Worker 已验证；Could 级法杖完整 UI 与部分 Effect 独立特效未实现，与第 8 章一致。

性能：动态图集提升千怪波 FPS 约 13.4%；五波实验整场均值约 34.48；对象池在同



图 7-1 游戏开始界面 (StartPanel)



图 7-2 三大关 DAG 跑图 (RunMapPanel)

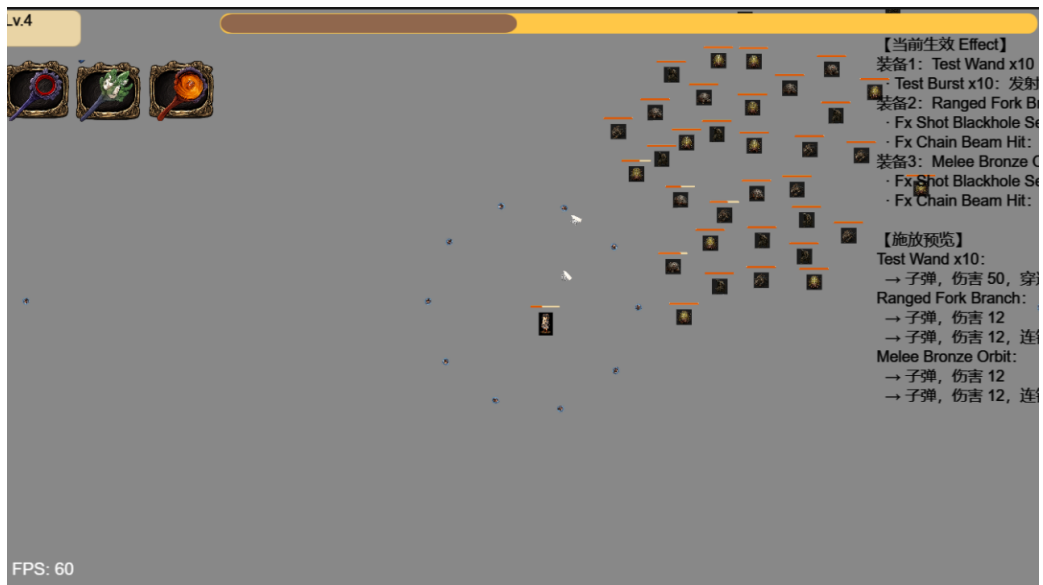


图 7-3 局内战斗（同屏多怪与弹幕）



图 7-4 Tab 暂停下的技能/Effect 装配面板

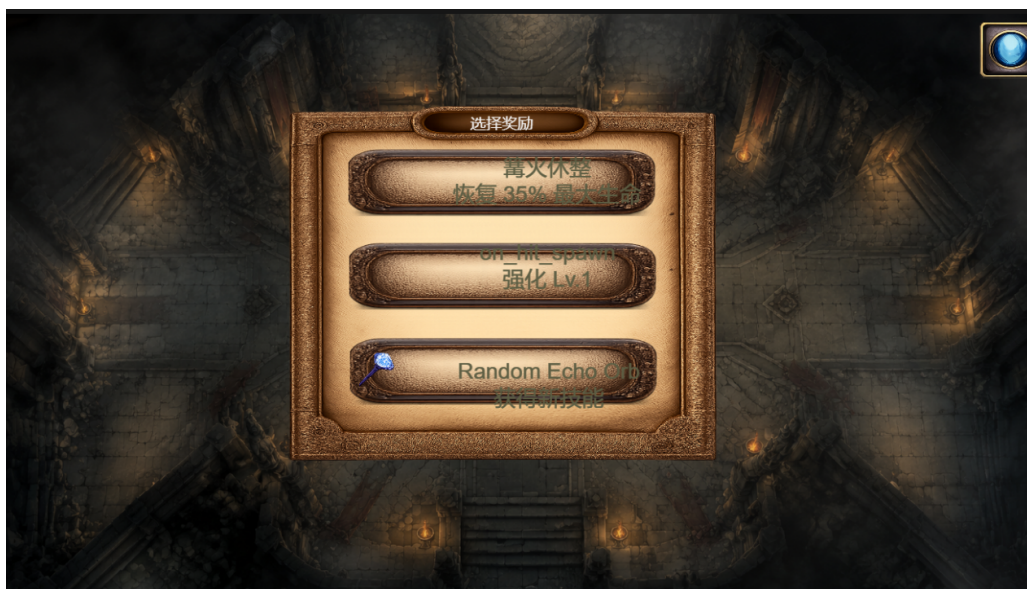


图 7-5 生存胜利后的三选一奖励面板



图 7-6 玩家死亡后的重启面板

窗消融下将 P95 降低约 36.5%。

遗留：配置未同步至 bin/ 时图标可能空白（已用脚本与 `isGameConfigReady` 缓解）；Worker 预热后同窗消融待补。

已知缺陷：部分 Effect 仅逻辑生效、无独立 VFX，已在答辩范围中如实标注。

7.7 本章小结

本章给出测试策略、单元验证、功能用例、环境与五波性能数据及解读；对象池价值已由第 4–5 波实测支持。截图就位后即可生成带插图的 PDF；Worker 同窗消融可作为后续实验补充。

第 8 章 总结与展望

8.1 工作总结

本文围绕 **Survivor And Fight**, 在 LayaAir 3.x 与 TypeScript 上完成了 Roguelite 生存战斗游戏的分析、设计、实现与测试。工作可归纳为以下方面, 并与各章对应:

- (1) 需求与选型 (第 1-2 章): 梳理国内外引擎架构、ECS、人群仿真与 H5 性能研究; 明确 LayaAir + 轻量 ECS + MVC UI + 配表驱动的技术路线。
- (2) 总体设计 (第 4 章): 提出分层架构与设计原则; 从 ECS Gameplay、MVC UI、Worker/对象池三方面说明系统结构; 给出端到端时序、可靠性、容错与基于组合的可扩展性策略。
- (3) 实现 (第 5 章): 以 SimpleEcsDemo 为组合根, 实现技能效果链、子弹碰撞、怪物 AI、元游戏跑图、Tab 装配与主循环; 典型场景与源码索引便于评阅对照。
- (4) 性能与测试 (第 6-7 章): 对象池显著降低千怪场景 P95 帧时间; 动态图集提升重度弹幕波次平均 FPS; 功能用例与单元脚本验证核心契约。

研究过程中参考 SyDRA 架构理解方法^[7]、Gregory 引擎分层论述^[7] 及 Unity 人群疏散实践^[7], 将模块化、重计算分离与 V&V 思想迁移到 H5 教学原型。系统可演示「菜单 → 跑图/选关 → 战斗 → 升级/装配 → 死亡重启」完整闭环, 达到毕业设计预期。

8.2 不足与展望

1. 内容与表现: 法杖三槽完整 UI、部分 Effect 独立特效与音效仍待完善;
2. 架构演进: ECS 可迁移至 Archetype/Chunk 存储以提升同屏规模^[7]; 碰撞可引入空间哈希等 broad phase;
3. 测试自动化: 补充 Worker「池 + Worker、已预热」同窗消融及端到端 UI 测试^[7];
4. 产品化: 局外存档、元进度与设置项 (帧率上限、特效降级);
5. 发布: 完善美术资源与适龄提示后的静态托管部署。

8.3 社会、健康与文化影响

本游戏为虚构战斗题材，采用抽象怪物与血条表现，应避免血腥写实。作为教学原型，未内置付费、社交诱导或敏感数据采集。若对外发布，须遵守网络游戏管理与未成年人保护规定，标注适龄提示与合理游戏时长；长时间游玩需注意视觉疲劳，Tab 暂停与跑图休息节点提供局内节奏缓冲^[7]。

8.4 绿色节能与兼容性

对象池、逻辑暂停与 Worker 卸载降低平均 CPU 占用，有利于终端续航与散热^[7]。Worker 与配表双通道回退提升浏览器兼容性；2D 渲染对 GPU 要求低于大型 3D 作品。未来可在设置中提供帧率上限与同屏怪数限制，进一步支撑低功耗模式。

8.5 本章小结

Survivor And Fight 展示了在 H5 环境下，以轻量 **ECS** + 数据驱动 + 可降级性能手段构建 Roguelite 原型的可行路径。论文叙述与仓库源码、测试数据相互对应；后续可在 Worker 定量验证、自动化测试与内容表现上继续完善，使原型从「可答辩演示」迈向「可发布小游戏」。

致 谢

在本论文完成之际，谨向指导教师致以衷心感谢。老师在选题、系统架构与论文撰写方面给予了耐心指导。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试与试玩中提供的帮助；感谢家人在毕业设计期间的理解与支持。

由于本人水平有限，文中疏漏之处恳请各位老师批评指正。

附 录

附录 A 名词术语及缩略词

术语/缩略语	含义
ECS	Entity Component System, 实体组件系统
Roguelite	轻 Roguelike, 含元进度与单局随机
DAG	Directed Acyclic Graph, 有向无环图
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件 (GBox/GButton 等)
Worker	Web Worker, 浏览器后台线程
MoSCoW	Must/Should/Could/Won't 需求优先级方法
V&V	Verification and Validation, 验证与确认
P95	第 95 百分位帧时间, 反映卡顿尖峰

附录 B 核心配表字段示例（节选）

Character 表: id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。玩家与怪物共用表结构，通过 roleType 区分预制体与默认数值。

Bullet 表: id、prefabPath、duration、speed、damage、penetration。子弹系统按 id 查表生成运动参数与伤害。

Skill 表: id、cooldown、effectSlotCount、图标路径等。一个技能可关联多行 SkillEffect。

SkillEffect 表: id、effect、damage、params、iconPath。effect 取值包括 bullet、modifier_split、modifier_chain、modifier_pierce、direct_damage。

完整 JSON 见项目 docs/config/ 目录;运行前须执行 tools/sync-config-to-bin.ps1 同步至 bin/config/。

附录 C 需求与测试用例映射（节选）

表 C-1 给出需求标识与测试用例、验证方式的对应关系，便于答辩时说明“需求—测试”可追溯性。

表 C-1 需求—测试映射（节选）

需求 ID	需求摘要	验证方式
R-ECS-01	实体销毁时清理全部组件	ecsCorePhase1.verify.ts
R-ECS-02	系统按分组与优先级顺序执行	ecsCorePhase1.verify.ts
R-SKILL-01	三装备栏独立冷却与自动施法	T-F-04，观察多技能轮替
R-SKILL-02	Tab 打开面板暂停战斗	T-F-05，GameSession.paused
R-SKILL-03	拖拽装配写回战斗实体	T-F-06，装配同步用例
R-BULLET-01	穿透/分裂/连锁按配表生效	T-F-04，高密度战斗观察
R-MON-01	波次刷怪与追逐分离	T-F-03，Worker 一致性对比
R-META-01	主菜单进入战斗与计时	T-F-09
R-MAP-01	跑图 DAG Boss 可达	100 次随机种子断言
R-PERF-01	对象池降低帧时间尖峰	表 7-6，第 4–5 波
R-PERF-02	动态图集提升千怪 FPS	表 7-4
R-UI-01	升级奖励三选一	T-F-07
R-UI-02	死亡重启面板	T-F-08

附录 D 缺陷记录（项目实测）

表 D-1 缺陷记录表

ID	描述	严重性	状态
B-01	配表未同步至 bin/config 导致技能图标空白	中	已修复（同步脚本）
B-02	Worker 脚本路径错误时回退主线程	低	已修复
B-03	部分 Effect 仅 UI 展示无独立 VFX	低	已知，论文已说明
B-04	法杖三槽运行时与专用 UI 未实现	中	规划项，第 8 章
B-05	千怪首次波次 FPS 低于预热后基线	低	已分析，见第 7 章